

IP addresses & subnets

Subnet mask :- This tells which part is the network portion and which part is host.

ex:- 192.168.1.25

Subnet mask

→ 255.255.255.0



These 3 nos. tells us the first 3

octets of this ip address is the network portion.

in other words those numbers will not change.

→ This is the host part which can change from 0-255, total of 256.

→ A device in this network (192.168.1) will have a total of 256 hosts which means in this network a total of 256 ip addresses can be assigned, so every device in this network gets an unique ip & the unique part will be the host part, so ex:- a device will have 192.168.1.5 as its ip address the other device will have 192.168.1.6 as its ip address in that network.

→ The four sections of ip address are called octets. Each octet = 8 bits

so 192.168.1.25

↓ ↓ ↓ ↓

8 bits 8 bits 8 bits 8 bits

$8+8+8+8 = 32 \text{ bits}$

Now Sometimes or most of the times an ip will be written as:-

192.168.1.25 /24

→ This is called CIDR Notation or prefix length. another way of writing the subnet mask.

/24 → represents the network part in total no. of bits, so $8+8+8 = 24$ which means in this ip the first 3 octets are network part

If its written as /16 → means $8+8$, so the first two octets represents network part & the last 2 ^{octets} can change (host part)

Depending on that we can determine which is the network portion & which is the host portion in an ip address.

ex:- /16 means → 255.255.0.0
/24 means → 255.255.255.0

(2)

The IPv4 consist of 4 billion ip addresses, at certain point we ran out of the ip addresses which is when NAT & private ip addresses were developed.

Instead of every device getting a public ip, each device in a network will receive private ips and the router on that network will receive one public ip assigned by the ISP. Now all devices in other networks can have same private ips and only the router will have a public ip.

If device wants to access a website it sends request, the router receives & translates the device's private ip to its public ip as source ip so other routers can route the packet when response as private ips ~~are not directly accessible by~~ cannot be reached directly from the internet.

This process of conversion of private $\rightarrow$ public or otherway is called Network Address Translation (NAT)

Even with that we still ran out of ip addresses which is when we discovered ipv6 (total of 340 undecillion addresses)

Most of the devices if using ipv4 have private ip addresses (check in cmd) But for devices with ipv6 addresses they are mostly public ipv6 addresses.

public ips :- Reachable across the internet & globally unique

Computers don't understand decimal ips like

192.168.1.21 → decimal

It understands binary, ex:-

11000000.10101000.00000001.00010101
↳ binary

How to convert?



128	64	32	16	8	4	2	1
(1)	(1)	0	0	0	0	0	0

pick number 192

~~128+64~~ 128 fits in so mark that 1

128+64 = 192 mark as 1

Mark the remaining 0 as we got the no. 192

so the binary of 192 → 11000000

lets do the 2nd octet 168
↓

(3)

128	64	32	16	8	4	2	1	(2 ⁰ to 2 ⁷ bits)
1	0	1	0	1	0	0	0	←

128 fits in mask 1

$128 + 64 = 192$ which is > 168 so doesn't count mark as 0

$128 + 32 = 160 < 168$ so counts mark as 1

$160 + 16 = 176 > 168$ so doesn't count

$160 + 8 = 168 = 168$ counts so mark as 1, we got the number so remaining mark as 0

binary version of 168 $\rightarrow 10101000$
we do the same for subnet mask

Subnet:- It means division within or of a network or dividing larger network into smaller networks

[Note:- We have total of 256 in a octet but from that 2 are reserved which leaves us 254 usable ip addresses]

Those two reserved for:-

- network address
- Broadcast address

last octet
↓
0
255

Subnetting:- changing the subnet mask value to manipulate with the no. of ip addresses to eventually get more networks from a larger network.

192.168.32.5 → ip
255.255.255.0 → subnet mask

CIDR

192.168.32.5/24

↓ decimal conversion

11000000. 1010 1000. 001 00000. 00000101
11111111. 1111 1111. 111 1111. ~~11111111~~
00000000

Here in subnet mask the 0's represent host bits

no. of bits = 8, $2^8 = 256$

= 254 (reserved addresses)
Usable
ips

Now lets say u need more ip addresses, in that case we manipulate the subnet mask value

~~1100~~

11111111. 11111111. 11111110. 00000000

Notice there's no extra 0. ~~110~~ And as we know '0' represent host bits. Now total no. of 0s = 9

$2^9 = 512$

= 510 usable ips

If need more ips just change subnet mask.

Also the decimal version will now look like

255.255.254.0

and CIDR →

192.168.32.5/23

The previous example was for simplified understanding, but subnetting means getting more networks not ips. (4)

So instead of $124 \rightarrow 123$

In subnetting $\rightarrow 124 \rightarrow 125 \rightarrow$ means now there's more networks

u can still increase the no. of ips, & its just that example will not be considered as subnetting.

Increasing host bits $\rightarrow$ supernetting

Increasing network bits $\rightarrow$ subnetting

Example:- lets say u have

$192.168.1.0/24$

u change it to

$192.168.1.0/26$ $\rightarrow$ this means the network bits increased which means u stole 2 bits from host part

1 bit can either be 0 or 1, so $2^1 = 2$

so for 2 bits = $2^2 = 4$

The combinations for those 2 borrowed bits

$\downarrow$

00
01
10
11

 $\rightarrow$ 4 possibilities

So that means we have 4 networks

and the new subnet mask:-

11111111.11111111.11111111.11000000

convert that to decimal



255.255.255.192



256 - 192 = 64

no. of host bits
↑
or $2^6 = 64$



This means every new sub-network (subnet) will start after 64 addresses

So → 0 - 63 (we count 0 as well)
64 - 127
128 - 191
192 - 255

So now the subnets are :-

192.168.1.0/26

192.168.1.64/26

192.168.1.128/26

192.168.1.192/26

we got more networks but less hosts bits which is 6, so $2^6 = 64$ which as we saw in example, there are 64 addresses per subnet.

4 networks $\times$ 64 = 256 (so its same just divided)

(5)
lets say in a situation u got 40 hosts per 3 coffee shops, and now u need to make subnets based on hosts.



Original network $\rightarrow 192.168.1.0/24$

to find how many host bits are needed



$2^5 = 32$ addresses (~~too~~ not enough)

$2^6 = 64$ addresses (enough) $\rightarrow$ usable 62

So we need 6 host bits

IPv4 has 32 bits in total = $32 - 6$
 $= 26$

Therefore subnet mask = $/26$

$/26 - /24 = 2$ so $2^2 = 4$ subnets

$192.168.1.0/26$ gives us some four subnets

$192.168.1.0/26$

$192.168.1.64/26$

$192.168.1.128/26$

we can use
these 3 subnets

$192.168.1.192/26 \rightarrow$ leave it unused

u got $172.16.0.0/20$

we need 6 branches with each needing

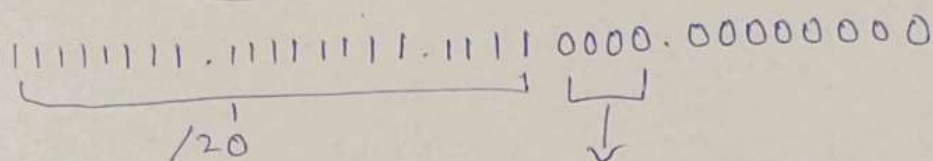
100 ips

answer
↓

First know the third octet change

172.16.0.0/20

$$\begin{array}{l} \text{subnet} = 20 \\ \times 32 - 20 \\ = 12 \\ \text{or} \end{array}$$



$$2^4 = 16$$

So the third octet will
range from 0-15

~~So~~ Now calculation

↓

$$2^7 = 128 \text{ ips (enough)}$$

$$32 - 7 = 25$$

new subnet mask = /25

$$\text{subtract with old} = 125 - 120 = 5$$

$$2^5 = 32 \text{ subnets}$$

we got 32 subnets each having 128 ips

Now remember the third octet originally
had 0-15 range, we always consider
the original subnet mask to list out subnets
which was /20.

So now, as we got 32 subnets with 128 ips each.

(6)

originally we had ~~32~~ 12 host bits, $2^{12} = 4096$ addresses

now $32 \times 128 = 4096$ so it matches up.

↓

172.16.0.0 → 172.16.0.127

172.16.0.128 → 172.16.0.255

~~We used all the 3rd octet~~
each subnet can contain 128 ips so here we run out, which means the next starts on

~~172.16.0.1~~

172.16.1.0 → 172.16.1.127

↓ continues
till

172.16.15.255

which means we had 16 separations
2 subnet each in one separation

$2 \times 16 = 32$ subnets

[Note:- Always consider the original
subnet mask when listing out subnets.]

Which concludes:-

To figure out the overall range subnets we refer the original subnet mask and to figure out the range of each subnets we refer the new subnet mask. So we need both.

All the subnets will have new subnet mask to determine this subnet ends in this range that's why after subnetting we write the new subnet mask & not the old one.

Reverse Subnetting

156.42.19.5/22

find network & broadcast address for this subnet

11111111.11111111.11111100.00000000

↓

$$2^2 = 4$$

Range will lie in

0-3, 4-7, 8-11, 12-15, 16-19, 20-23

The third octet value 19 falls in 16-19 range so pick the 1st for network & last for broadcast and for the fourth octet as all was bits 0 we pick 0 as network & 255 as broadcast.

↓ ↓

156.42.16.0 → network address

156.42.19.255 → broadcast address

Subnetting a subnet

7

lets say u got a network 172.21.42.0/24



u need 4 more networks with requirements

10 hosts

57 hosts

26 hosts

117 hosts

here we will be using VLSM

always choose the largest one first

<u>so</u> <u>hosts</u>	<u>minimum address</u>	<u>Subnet</u>
117	128	/25
57	64	/26
26	32	/27
10	16	128

172.21.42.0/25

↳ This makes two subnets as $124 \rightarrow /25 = 1$
 $= 2^1 = 2$

172.21.42.0/25 (0-127) → This one allocated for 117 hosts

172.21.42.128/25 (128-255)

↳ Now we further subnet this second one for other hosts requirements.

57 hosts



172.21.42.128 / 26 $\xrightarrow{\text{range}}$

172.21.42.128 - 172.21.42.191 (64) $\rightarrow$ used for 57 hosts

172.21.42.192 - 172.21.42.255 (64)

[$\rightarrow$ This second one will be further subdivided for the remaining hosts.

at the end



26 hosts



172.21.42.192 / 27

10 hosts



~~172.41~~

172.21.42.224 / 28